Refinamento Técnico — AgendaPet Web

Como construir: arquivos, responsabilidades, contratos de dados, fluxos, modelos de código e plano de testes

Desenvolvimento Front-End [React] T4 | Módulo 1 | Semanas 13 e 14 | Projeto: **Mini-Projeto Avaliativo M1.2 (AgendaPet Web)**
Leia o **Documento Funcional** antes (ele diz *o quê* e *por quê*). Aqui está o **como**. Os modelos de código são **esqueletos para você completar** (procure os `TODO`): não são a solução pronta. Tudo o que vier de IA precisa ser revisado e explicado por você (veja a seção 14).

---

1. Visão técnica

Item	Decisão
Linguagens	HTML, CSS e JavaScript puros (sem bibliotecas nem frameworks) + Node.js 20+ para os scripts de terminal
Módulos	**ESM** (`"type": "module"` no `package.json`). Um script em **CommonJS** (`.cjs`) para demonstrar a diferença
Servidor local	`http-server` , instalado como **devDependency local**; roda com `npm start` (porta 5500, sem cache)
Persistência	`localStorage` (somente texto: `JSON.stringify` / `JSON.parse`), todas as chaves com prefixo `agendapet:`
APIs externas	**ViaCEP** (endereço) e **BrasilAPI** (feriados), ambas sem chave e com CORS liberado
Princípio central	**Um arquivo, uma responsabilidade.** E a regra de negócio vive em `js/dominio/` , **sem DOM**, para rodar na página e no Node

```
index.html ── <script type="module"> ──► js/app.js  (maestro: estado + eventos)
                                          ├─► js/ui.js        desenha o DOM
                                          ├─► js/api.js       fetch ──► servidor local · ViaCEP · BrasilAPI
                                          ├─► js/storage.js   localStorage
                                          └─► js/dominio/*    regras puras (classes, validações, fechamento)
                                                   ▲
                        scripts/fechamento-dia.js (ESM) e scripts/validar-agenda.cjs (CJS) usam o mesmo
domínio
```

2. Estrutura de arquivos

```
agendapet-web/
├── index.html
├── package.json · package-lock.json · .gitignore   (node_modules/ no .gitignore)
├── README.md
├── css/style.css
```

```
├── data/
│   ├── servicos.json          (≥ 6 serviços)
│   ├── agenda-exemplo.json    (≥ 8 agendamentos, usados pelos scripts de Node)
│   └── agenda-invalida.json   (erros de propósito, para provar o validador)
├── img/ logo.svg · favicon.svg
├── js/
│   ├── app.js · ui.js · api.js · storage.js
│   └── dominio/ agendamento.js · regras.js · fechamento.js
├── scripts/ fechamento-dia.js · validar-agenda.cjs
├── tests/dominio.test.js      (opcional: bônus)
├── .vscode/launch.json
├── .github/pull_request_template.md
├── planejamento/sprint-planning.md
└── docs/ kanban.png · depuracao.png · bug-report.md
```

3. Responsabilidade de cada arquivo (e o que NÃO fazer)

Arquivo	Faz	NÃO faz
`index.html`	Estrutura semântica, rótulos, ids dos campos, `<script type="module" src="js/app.js">`	Lógica, `onclick=""` no HTML, estilos inline
`css/style.css`	Variáveis, layout Flexbox, responsividade, estados, tema escuro	`!important`, seletores de id, seletores de tag aninhados que "brigam" com classes
`js/dominio/agendamento.js`	`CONFIG`, `STATUS`, classes `Servico`, `Agendamento`, subclasse domiciliar, fábrica, closure do protocolo	Tocar em `document`, `window`, `localStorage` ou `fetch`
`js/dominio/regras.js`	Datas/horas, funcionamento, validação do formulário (devolve `{campo: mensagem}`), conflito, feriado, moeda	Mostrar mensagens na tela
`js/dominio/fechamento.js`	Ordenar, filtrar (callbacks), próximo atendimento, `calcularFechamento`, linhas do relatório	DOM, storage, rede
`js/storage.js`	**Único** lugar que chama `localStorage`; `try/catch`; valores padrão; limpar só o prefixo	Regras de negócio
`js/api.js`	**Único** lugar que chama `fetch`; `response.ok`; `ErroApi` com `tipo`; cache de feriados	Mexer no DOM
`js/ui.js`	Desenhar: `createElement`, `textContent`, `classList`; os 3 estados; toast (`setTimeout`)	Buscar dados, salvar, decidir regras
`js/app.js`	Estado, `addEventListener`, orquestra os outros módulos, `setInterval`	Regra de negócio dentro dos handlers; `innerHTML` com dado do usuário
`scripts/fechamento-dia.js`	ESM: lê os JSONs com `fs/promises`, usa o domínio, imprime o relatório; `--data=` e `--arquivo=`	Reescrever regra que já existe no domínio
`scripts/validar-agenda.cjs`	CommonJS: `require` de `fs` / `path`, `await import()` do domínio, lista de erros, `process.exit(1)`	Usar `import` estático (não funciona em CJS)

4. Contratos de dados

4.1 Chaves do localStorage

Chave	Conteúdo	Quando lê / grava
`agendapet:agenda`	Lista de **objetos simples** (veja 4.3), não instâncias de classe	Lê na carga (depois do catálogo); grava a cada mudança
`agendapet:filtro`	`"todos"` | `"Agendado"` | `"Concluído"` | `"Cancelado"`	Lê na carga; grava ao mudar o filtro
`agendapet:tema`	`"claro"` | `"escuro"`	Lê na carga; grava ao trocar o tema
`agendapet:feriados:2026`	`{ "salvoEm": 1760000000000, "feriados": [ { "date": "2026-12-25", "name": "Natal" } ] }` — uma chave por ano	Lê antes de chamar a BrasilAPI; vale 30 dias

> **Cuidado**
> O `localStorage` só guarda **texto**. Instância de classe vira `{}` no `JSON.stringify` (perde os métodos). Por isso a classe tem `paraJSON()` (gravar) e existe uma **fábrica** `criarAgendamento()` (restaurar). Na primeira visita `getItem` devolve `null` : sempre tenha um valor padrão. E o dado pode estar **corrompido**: `JSON.parse` dentro de `try/catch` .

4.2 Catálogo (data/servicos.json)

```
[
  { "id": "banho", "nome": "Banho", "descricao": "Banho com shampoo neutro, secagem e perfume.",
    "preco": 60, "duracaoMin": 60, "domiciliar": true },
  { "id": "adestramento", "nome": "Adestramento (sessão)", "descricao": "…",
    "preco": 110, "duracaoMin": 60, "domiciliar": false }
]
```

4.3 Agendamento (objeto simples: o que vai para o localStorage e para o JSON exportado)

```
{
  "protocolo": "AP-0002", "tutor": "Carlos Lima", "telefone": "(11) 97766-5544",
  "pet": "Luna", "especie": "Cão", "servicoId": "banho-tosa",
  "data": "2026-10-20", "hora": "09:00",           // data "AAAA-MM-DD" e hora "HH:MM" (texto)
  "status": "Concluído",                           // "Agendado" | "Concluído" | "Cancelado"
  "observacoes": "Tosa na máquina.",
  "domiciliar": true,                              // false → "endereco": null
  "endereco": { "cep": "01310-200", "logradouro": "Avenida Paulista", "numero": "1578",
                "complemento": "apto 42", "bairro": "Bela Vista", "cidade": "São Paulo", "uf": "SP" }
}
```

> **Por que data e hora em texto?**
> "2026-10-20" e "09:00" ordenam e comparam corretamente como texto (`"09:00" < "10:00"`). Cuidado: `new Date("2026-10-20")` é interpretado como UTC e pode virar o dia anterior no Brasil. Para montar uma data local use `new Date("2026-`

> `10-20T09:00:00")` (sem o "Z").

4.4 Objetos em memória (classes)

Classe	Atributos	Métodos / getters
`Servico`	`id, nome, descricao, preco, duracaoMin, domiciliar`	—
`Agendamento`	`protocolo, tutor, telefone, pet, especie, servico` (objeto Servico), `data, hora, status, observacoes`	`get ativo`, `get inicio`, `get tipo`, `calcularValor()`, `resumo()`, `conflitaCom(outro)`, `alterarStatus(novo)`, `paraJSON()`
`AgendamentoDomiciliar extends Agendamento`	`+ endereco`	sobrescreve `calcularValor()` (`super` + taxa), `resumo()`, `tipo`, `paraJSON()`; novos: `taxaDeslocamento()`, `enderecoCompleto()`

Por que herança aqui? O domiciliar *é um* agendamento (mesmo tutor, pet, serviço, data, hora, status), mas tem dado a mais (endereço) e valor diferente (taxa). Só o que muda é reescrito. Você precisa saber justificar isso na apresentação ao vivo.

4.5 Estado do app (js/app.js): uma única "fonte da verdade"

```
const estado = {
  servicos: [],              // Servico[]
  agenda: [],                // Agendamento[] (e AgendamentoDomiciliar)
  dataAgenda: hojeISO(),     // dia exibido na agenda
  filtroStatus: 'todos',
  busca: '',
  expandidos: new Set(),     // protocolos com detalhes abertos
  feriado: null,             // { date, name } se a data escolhida for feriado
  gerarProtocolo: criarGeradorProtocolo(0),   // closure
  timerLembretes: null,      // id do setInterval (só pode existir UM)
  lembretesAtivos: true,
};
```

5. As APIs

5.1 Servidor local (catálogo)

`GET data/servicos.json` → array JSON. Caminho **relativo** à página. Para simular erro, aponte para um arquivo que não existe: o servidor responde **404** e `response.ok` é `false` (o `fetch` **não** rejeita em 404!).

5.2 ViaCEP — endereço por CEP

Item	Detalhe
Requisição	`GET https://viacep.com.br/ws/{cep}/json/` — `{cep}` com 8 dígitos, sem hífen
Sucesso	`{ "cep": "01001-000", "logradouro": "Praça da Sé", "complemento": "lado ímpar", "bairro": "Sé", "localidade": "São Paulo", "uf": "SP", … }`
CEP bem formatado, mas inexistente	HTTP **200** com `{ "erro": true }` (ou `"erro": "true"`) — **você precisa checar** `dados.erro`
CEP malformado	HTTP 400 — por isso valide os 8 dígitos **antes** de chamar
Mapeamento para o formulário	`localidade → cidade`, `logradouro → rua`, `bairro`, `uf`

5.3 BrasilAPI — feriados nacionais

Item	Detalhe
Requisição	`GET https://brasilapi.com.br/api/feriados/v1/{ano}`
Resposta	`[ { "date": "2026-12-25", "name": "Natal", "type": "national" }, … ]`
Uso	`lista.find((f) => f.date === "2026-12-25")` — a data do input `type="date"` já vem como `"AAAA-MM-DD"`
Cache	Guarde `{ salvoEm: Date.now(), feriados }` em `agendapet:feriados:{ano}`; use se `Date.now() - salvoEm < 30 dias`

5.4 Tipos de erro (ErroApi) e como a tela reage

tipo	Quando	Mensagem sugerida
`formato`	CEP sem 8 dígitos (nem chama a rede)	O CEP deve ter 8 dígitos.
`nao-encontrado`	ViaCEP devolveu `erro`	CEP não encontrado. Preencha o endereço manualmente.

`rede`	`fetch` rejeitou (sem internet)	Sem conexão… Preencha manualmente.
`http`	`response.ok` falso	O serviço … está indisponível agora.

6. Fluxos principais

6.1 Abrir a página

1. `iniciar()`: aplica o tema salvo, lê o filtro, define a data mínima do formulário, liga os eventos, inicia os lembretes.
2. `carregarCatalogo()`: mostra **carregando** → `await carregarServicos()` → em erro mostra a mensagem com "Tentar novamente"; em sucesso renderiza os serviços e popula o `<select>`.
3. Com o catálogo pronto: `restaurarAgenda()` (objetos simples → instâncias via fábrica; item inválido é ignorado) e o gerador de protocolo recomeça do maior número salvo.
4. `atualizarTela()`: agenda, fechamento e relógio.

6.2 Salvar um agendamento (submit)

1. `evento.preventDefault()`; limpar erros antigos; ler o formulário (`lerFormulario()`).
2. `validarAgendamento(dados, servico)` → objeto de erros por campo. Se for feriado e não confirmou, adicionar o erro.
3. Sem erros: criar o agendamento pela fábrica e checar `temConflito(estado.agenda, novo)`.
4. Se há erro: `mostrarErros` (marca `aria-invalid`, escreve a mensagem, **foca o primeiro inválido**) e para.
5. Sem erros: gerar o protocolo, `push`, `salvarAgenda()`, mostrar o dia do novo item, limpar o formulário, `atualizarTela()`, toast.

6.3 Digitar o CEP (US04)

1. No evento `input`: se já tem 8 dígitos, chama `consultarCep()`; se tem menos, limpa a mensagem.
2. Evita repetir a consulta do mesmo CEP (`ultimoCep`).
3. `await buscarCep(cep)` → preenche rua, bairro, cidade e UF e leva o foco ao número. **Descarte a resposta** se, enquanto esperava, o CEP digitado mudou (corrida de requisições).
4. Qualquer `ErroApi` vira a mensagem do item 5.4 e o preenchimento manual continua liberado.

6.4 Escolher a data (US05)

1. Evento `change` da data → limpa o aviso → `buscarFeriados(ano)` (cache primeiro).
2. Se a data está na lista: mostra o nome e o checkbox "Agendar mesmo assim". Se a API falhou: mostra o aviso de indisponibilidade (sem checkbox).
3. Se a pessoa trocou a data durante a espera, descarte o resultado.

6.5 Clicar na lista (delegação de eventos)

1. **Um** `addEventListener('click')` na `<ul>` (os itens são recriados a cada renderização!).
2. `evento.target.closest('[data-acao]')` descobre qual botão; `closest('.item').dataset.protocolo` descobre qual item.
3. Um `switch` trata `alternar`, `concluir` e `cancelar`; depois salva, redesenha e **devolve o foco** ao item (senão quem usa teclado se perde).
4. Mouse: `mouseover` / `mouseout` também por delegação, com `closest('.item')` (eles "borbulham" dos filhos).

6.6 Relógio e lembretes (a cada 30 s)

1. `iniciarLembretes()` começa com `clearInterval(estado.timerLembretes)` (**nunca dois timers**) e então `setInterval(atualizarRelogio, 30_000)`.
2. `atualizarRelogio()`: mostra a hora; `proximoAtendimento(...)` do dia de hoje; se faltam ≤ 15 min e os lembretes estão ativos, mostra o aviso.
3. Só redesenhe a lista se mudou **quem está atrasado** (redesenhar à toa a cada 30 s tira o foco de quem navega por teclado).
4. Pausar: `clearInterval`, guarda `null`, atualiza o botão (`aria-pressed`).

6.7 Fechar o dia pelo terminal

1. `node scripts/fechamento-dia.js --data=2026-10-20`
2. Lê `servicos.json` e `agenda-exemplo.json` (`fs/promises`) → `criarAgendamento` para cada item → `calcularFechamento` → `montarLinhasRelatorio` → `console.log`.
3. Data inválida ou arquivo ruim: mensagem clara e `process.exitCode = 1`.

7. Tratamento do "caminho triste"

Falha	O que a pessoa vê	O que NÃO pode acontecer
Catálogo não carrega (404, rede)	Mensagem de erro + "Tentar novamente"; formulário e agenda seguem utilizáveis onde possível	Tela vazia, exceção não tratada no console
ViaCEP fora do ar / sem rede / CEP inexistente	Mensagem específica; campos de endereço continuam editáveis	Bloquear o agendamento
BrasilAPI fora do ar	Aviso "não foi possível verificar feriados" e o agendamento prossegue	Exigir confirmação de um feriado que não conhecemos
`localStorage` corrompido ou vazio	Página abre com agenda vazia e valores padrão	`JSON.parse` lançando erro; `null.map`
Item inválido dentro da agenda salva (ex.: serviço que não existe mais)	O item é ignorado; o resto carrega	Toda a agenda sumir por causa de um item
Resposta da API chega depois que a pessoa mudou o CEP/data	Resposta atrasada é descartada	Endereço/feriado de um valor antigo aparecendo na tela
Vários cliques em "Concluir"	Botões desabilitados após o primeiro	Alterar duas vezes o mesmo agendamento
Script de Node com arquivo inexistente ou JSON inválido	Mensagem clara no terminal e código de saída 1	Stack trace gigante sem explicação

8. Modelos de código (esqueletos para completar)

Não copie sem entender: cada bloco traz `TODO` e uma dica. As regras (preço, taxa, funcionamento…) **você decide** e registra no README.

8.1 package.json (US11)

```
{
  "name": "agendapet-web",
  "version": "1.0.0",
  "description": "TODO: uma frase sobre o projeto",
  "type": "module",
  "scripts": {
    "start": "http-server . -p 5500 -c-1 -o",
    "fechamento": "node scripts/fechamento-dia.js --data=2026-10-20",
    "validar": "node scripts/validar-agenda.cjs"
  }
```

```
}
// Depois, no terminal:  npm install --save-dev http-server   (cria devDependencies + package-lock.json)
```

8.2 Classes, herança, this e closure (US07) — js/dominio/agendamento.js

```
export const CONFIG = Object.freeze({ abertura: '08:00', fechamento: '18:00', passoMin: 30 /* TODO: taxas,
cidade da loja… */ });

export class Agendamento {
  constructor({ protocolo, tutor, /* TODO: demais campos */ servico, data, hora, status = 'Agendado' }) {
    this.protocolo = protocolo;
    // TODO: guardar os demais campos em this.*
  }

  get ativo() { /* TODO: true quando o status NÃO é "Cancelado" */ }

  calcularValor() { return this.servico.preco; }

  conflitaCom(outro) {
    // TODO: true se os DOIS estão ativos, são de datas/horas iguais e NÃO são o mesmo objeto (this !== outro)
  }

  paraJSON() { /* TODO: objeto simples; em vez de this.servico, guarde servicoId: this.servico.id */ }
}

export class AgendamentoDomiciliar extends Agendamento {
  constructor({ endereco, ...dados }) {
    super(dados);              // 1) chama o construtor da classe-mãe  2) só depois use this
    this.endereco = endereco;
  }
  calcularValor() { return super.calcularValor() + this.taxaDeslocamento(); }
  taxaDeslocamento() { /* TODO: regra da taxa (compare cidades ignorando acentos/maiúsculas) */ }
}

// Fábrica: decide a classe a partir do objeto simples (necessária para restaurar do localStorage)
export function criarAgendamento(dados, catalogo) { /* TODO: achar o serviço pelo id; domiciliar ? new
AgendamentoDomiciliar(...) : new Agendamento(...) */ }

// CLOSURE: `contador` fica "presa" na função devolvida (sem variável global)
export function criarGeradorProtocolo(ultimo = 0) {
  let contador = ultimo;
  return function proximoProtocolo() {
    /* TODO: somar 1 e devolver "AP-0001" (dica: String(n).padStart(4, '0')) */
  };
}
```

8.3 Callback e métodos de array (US06/US07) — js/dominio/fechamento.js

```
// CALLBACK: recebe FUNÇÕES (critérios). `every` exige que o item passe em TODOS.
export function filtrarAgendamentos(lista, ...criterios) {
```

```
  // TODO: return lista.filter(item => criterios.every(criterio => criterio(item)));
}
// Fábrica de critérios: devolve uma função pronta para usar como callback
export const porStatus = (status) => (agendamento) => { /* TODO: 'todos' passa; senão compare o status */ };

export function calcularFechamento(lista, data) {
  const doDia = lista.filter((a) => a.data === data);
  const concluidos = /* TODO: filter por status 'Concluído' */ [];
  // reduce: soma o valor SÓ dos concluídos  →  faturamento
  // reduce: agrupa por serviço { id: { servico, quantidade, faturado } }  →  Object.values(...).sort(...)
  // every: o dia está encerrado se existe atendimento e NENHUM está 'Agendado'
  // some: existe atendimento domiciliar?
  // find: (em proximoAtendimento) primeiro 'Agendado' a partir da hora atual
  return { /* TODO */ };
}
```

8.4 fetch com os 3 estados e erros por tipo (US02, US04) — js/api.js

```
export class ErroApi extends Error {
  constructor(tipo, mensagem) { super(mensagem); this.tipo = tipo; }   // 'rede' | 'http' | 'formato' | 'nao-encontrado'
}

async function buscarJSON(url) {
  let resposta;
  try {
    resposta = await fetch(url);                       // só REJEITA se a requisição nem chegou ao servidor
  } catch {
    throw new ErroApi('rede', 'Sem conexão.');
  }
  if (!resposta.ok) throw new ErroApi('http', `Erro HTTP ${resposta.status}`);   // 404, 500… NÃO rejeitam o fetch
  return resposta.json();                              // também é uma Promise: use await
}

// ViaCEP: depois de buscarJSON, confira  if (dados.erro)  → ErroApi('nao-encontrado')   (HTTP 200 com { "erro": true })
```

```
// app.js — os 3 estados
async function carregarCatalogo() {
  ui.mostrarEstado(aviso, 'carregando', 'Carregando serviços…');
  try {
    const dados = await carregarServicos();
    /* TODO: guardar em estado.servicos (new Servico) e renderizar */
  } catch (erro) {
    ui.mostrarEstado(aviso, 'erro', erro.message, carregarCatalogo);   // 3º argumento: botão "Tentar novamente"
    return;
  }
  ui.esconderEstado(aviso);
}
```

8.5 localStorage seguro (US08) — js/storage.js

```
const PREFIXO = 'agendapet:';

export function lerJSON(chave, padrao) {
  try {
    const bruto = localStorage.getItem(chave);        // null na 1ª visita
    return bruto === null ? padrao : JSON.parse(bruto);
  } catch {
    return padrao;                                    // JSON corrompido não derruba a página
  }
}
export function salvarJSON(chave, valor) { /* TODO: try { setItem(chave, JSON.stringify(valor)); return true }
catch { return false } */ }

export function limparDados() {
  // TODO: pegar Object.keys(localStorage), filtrar as que começam com PREFIXO e remover só essas
}
```

8.6 Delegação de eventos e atalhos de teclado (US06) — js/app.js

```
$('lista-agenda').addEventListener('click', (evento) => {
  const botao = evento.target.closest('[data-acao]');          // qual botão?
  if (!botao) return;
  const protocolo = botao.closest('.item').dataset.protocolo;  // qual item?
  switch (botao.dataset.acao) {
    case 'concluir': /* TODO */ break;
    case 'cancelar': /* TODO */ break;
    case 'alternar': /* TODO: abrir/fechar detalhes (aria-expanded) */ break;
  }
});

document.addEventListener('keydown', (evento) => {
  if (evento.key === 'Escape' && evento.target.id === 'busca') { /* TODO: limpar a busca */ return; }
  if (evento.target.matches('input, textarea, select')) return;   // não roube as teclas de quem está
digitando!
  if (evento.key === '/') { evento.preventDefault(); /* TODO: focar a busca */ }
  if (evento.key.toLowerCase() === 'n') { /* TODO: ir ao formulário e focar o campo tutor */ }
});
```

8.7 Timers (US09)

```
// setInterval: sempre guarde o id e limpe antes de criar outro
function iniciarLembretes() {
  clearInterval(estado.timerLembretes);
  estado.timerLembretes = setInterval(atualizarRelogio, 30_000);
  atualizarRelogio();                       // já mostra algo agora, sem esperar 30 s
}
```

```
// setTimeout: um toast por vez; um novo cancela o timer do anterior (senão o toast novo some cedo demais)
let timerToast = null;
export function mostrarToast(texto, tipo = 'sucesso', duracaoMs = 4000) {
  clearTimeout(timerToast);
  // TODO: preencher #toast com textContent, mostrar (hidden = false)
  timerToast = setTimeout(() => { /* TODO: esconder */ }, duracaoMs);
}
```

8.8 Script ESM de terminal (US11) — scripts/fechamento-dia.js

```
import { readFile } from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { Servico, criarAgendamento } from '../js/dominio/agendamento.js';   // MESMOS módulos da página
// import { calcularFechamento, montarLinhasRelatorio } from '../js/dominio/fechamento.js';

// ESM não tem __dirname: calcule a partir de import.meta.url
const raiz = path.resolve(path.dirname(fileURLToPath(import.meta.url)), '..');
const lerJSON = async (rel) => JSON.parse(await readFile(path.join(raiz, rel), 'utf8'));

// Argumentos:  node scripts/fechamento-dia.js --data=2026-10-20
const arg = process.argv.find((a) => a.startsWith('--data='));
// TODO: validar a data; carregar catálogo e agenda; calcular; imprimir; em erro → console.error +
process.exitCode = 1
```

8.9 Script CommonJS com import() dinâmico (US11) — scripts/validar-agenda.cjs

```
'use strict';
const fs = require('node:fs');          // CommonJS: require()
const path = require('node:path');      // e __dirname existe aqui
const raiz = path.resolve(__dirname, '..');

async function main() {
  const agenda = JSON.parse(fs.readFileSync(path.resolve(raiz, process.argv[2] || 'data/agenda-exemplo.json'),
'utf8'));

  // Importar ESM a partir de CJS: SÓ com import() dinâmico (devolve Promise → await)
  const regras = await import('../js/dominio/regras.js');

  const erros = [];
  // TODO: para cada item → campos obrigatórios, status válido, regras.dataValida / horaValida, serviço
existente,
  //       horário dentro do funcionamento, endereço se domiciliar; depois conflitos de horário (laço duplo).
  if (erros.length) {
    console.error(`[ERRO] ${erros.length} problema(s):`);
    erros.forEach((e) => console.error('  - ' + e));
    process.exit(1);                    // código de saída ≠ 0 = falha (automações enxergam isso)
  }
  console.log('[OK] agenda válida');
```

```
}
main();
```

8.10 .vscode/launch.json (US12)

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Depurar fechamento do dia (Node)",
      "type": "node",
      "request": "launch",
      "program": "${workspaceFolder}/scripts/fechamento-dia.js",
      "args": ["--data=2026-10-20"],
      "console": "integratedTerminal",
      "skipFiles": ["<node_internals>/**"]
    }
    // TODO: uma configuração para o validar-agenda.cjs (args: ["data/agenda-invalida.json"])
  ]
}
```

8.11 Teste com o executor nativo (bônus) — tests/dominio.test.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { criarGeradorProtocolo } from '../js/dominio/agendamento.js';

test('closure: o protocolo é sequencial', () => {
  const proximo = criarGeradorProtocolo(7);
  assert.equal(proximo(), 'AP-0008');
  assert.equal(proximo(), 'AP-0009');
});
// Rode:  npm test   →  "test": "node --test tests/"
```

8.12 Padrões de branch, commit e Pull Request

```
Branch:  feat/US03-formulario        fix/US07-faturamento-concluidos      chore/setup-projeto      docs/readme
Commit:  feat(US03): adiciona validação de data e horário
         fix(US07): soma só atendimentos concluídos no faturamento
         docs: descreve a regra da taxa domiciliar
         (tipo(ticket): descrição no imperativo — feat, fix, docs, refactor, test, chore)
Pull Request: preencha o modelo de .github/pull_request_template.md (o que mudou, ticket, como testar,
evidência, checklist)
```

9. Node.js na prática: npm, global vs local, ESM vs CommonJS

Quero…	Comando
Criar o projeto	`npm init -y` (depois edite nome, descrição, `"type": "module"` e scripts)
Instalar uma ferramenta **local** (dev)	`npm install --save-dev http-server` → vai para `devDependencies` e `package-lock.json`
Instalar tudo do projeto (após clonar)	`npm install`
Ver o que está instalado	`npm ls` (local) · `npm ls -g --depth=0` (global)
Instalar **global** (só para entender)	`npm install -g http-server` — fica na máquina, **não** entra no projeto
Rodar um pacote sem instalar	`npx http-server --help`
Rodar um script do `package.json`	`npm run fechamento` (`start` e `test` dispensam o `run`)

	CommonJS	ESM
Importar / exportar	`const x = require('./x')` · `module.exports = …`	`import x from './x.js'` · `export function …` (a extensão `.js` é obrigatória no import)
Como o Node decide	`.cjs`, ou `.js` sem `"type": "module"`	`.mjs`, ou `.js` com `"type": "module"`
`__dirname`	Existe	Não existe: `fileURLToPath(import.meta.url)`
Navegador	Não entende	Entende (`<script type="module">`)
Um carrega o outro	CJS → ESM só com `await import()`	ESM → CJS com `import` normal

> **Por que ESM é o padrão deste projeto**
> É o único formato que roda **nos dois lugares**: o navegador e o Node. Assim `js/dominio/` é escrito uma vez. O CommonJS entra só no validador, para você enxergar e explicar a diferença.

10. Depuração no VS Code (US12)

1. Abra `js/dominio/fechamento.js` e clique na margem esquerda da linha desejada: aparece o **ponto vermelho** (breakpoint).
2. Pressione **F5** e escolha a configuração do `launch.json`. A execução **pausa** na linha.

3. Painel **Variables**: valores no momento. Painel **Watch**: adicione expressões (ex.: `concluidos.length`). **Call Stack**: quem chamou quem.
4. **F10** (Step Over) executa a linha; **F11** (Step Into) entra na função; **Shift+F11** sai; **F5** continua.
5. Clique direito no breakpoint → **Editar** → **Condição** (ex.: `a.status === 'Agendado'`) ou **Logpoint** (escreve no Debug Console sem parar).
6. Documente em `docs/bug-report.md`: título, passos, esperado × obtido, causa raiz, evidência (print em `docs/depuracao.png`) e correção (commit/PR).

> **Sobre o bug**
>
> Use um bug **real** que apareceu no seu desenvolvimento. Se nenhum surgir, crie uma branch, introduza um erro controlado (por exemplo, o faturamento somar atendimentos cancelados ou agendados), reproduza, depure, corrija e documente. O que importa é mostrar o **método**: reproduzir → hipótese → evidência no depurador → causa raiz → correção.

11. CSS: Flexbox, mobile-first e especificidade

Tema	Orientação
Mobile-first	Escreva primeiro o CSS do celular (uma coluna). Use `@media (min-width: 640px)` e `(min-width: 1024px)` para **acrescentar** ao layout maior
Flexbox	Coluna no celular (`flex-direction: column`); em ≥ 1024 px o painel vira `row` com duas colunas (`flex: 1 1 380px` e `flex: 2 1 560px`); `flex-wrap` nas linhas de campos; `min-width: 0` nas colunas para o conteúdo não estourar
Variáveis	Cores e medidas em `:root` ; tema escuro em `:root[data-tema="escuro"]` redefinindo só as variáveis
Especificidade	Classes simples (0,1,0). Seletor de tag + classe (`.agenda button` = 0,1,1) **vence** uma classe sozinha e causa "o estilo não pega". Dê uma classe própria ao elemento. Sem `!important` e sem `#id` no CSS
`hidden`	`[hidden]` perde para `display: flex` de uma classe (mesma especificidade, vale a ordem). Declare `[hidden] { display: none; }` **por último** no arquivo
Foco	`:focus-visible` com contorno de 3 px e `outline-offset` . Nunca `outline: none` sem substituto
Toque	Botões e campos com `min-height: 44px` . O toast deve ter `pointer-events: none` (senão cobre botões embaixo)

12. Passo a passo sugerido (um ticket por branch)

Quando	Branch	O que fazer	Commits de exemplo
16/10	`chore/setup-projeto`	Repositório, Kanban (12 cartões), estimar e escrever `sprint-planning.md` ; `npm init` , `.gitignore` , modelo de PR	`chore: inicia projeto e estrutura de pastas` `docs: adiciona planejamento da sprint`
20/10	`feat/US01-US10-html-css`	HTML semântico com todos os ids; CSS mobile-first com variáveis, Flexbox, foco, tema	`feat(US01): cria estrutura semântica da página` `feat(US10): aplica layout responsivo com Flexbox`
20–22/10	`feat/US02-US03-catalogo-formulario`	`servicos.json` + `api.js` (3 estados); `regras.js` (validação); submit com erros acessíveis	`feat(US02): carrega catálogo com fetch e 3 estados` `feat(US03): valida formulário e mostra erros na tela`

22/10	`feat/US04-US05-apis`	ViaCEP e BrasilAPI com erros por tipo, resposta atrasada descartada e cache de feriados	`feat(US04): preenche endereço pelo CEP` `feat(US05): alerta feriados com cache`
23/10	`feat/US06-US07-agenda-dominio`	Classes, herança, conflito, protocolo (closure), callbacks, fechamento; lista, busca, filtros, atalhos. **Teste o domínio no Node antes de ligar na tela**	`feat(US07): cria classes com herança e fechamento do dia` `feat(US06): lista agenda com busca e atalhos`
27/10	`feat/US08-US09-storage-timers`	`storage.js`, exportação, limpar dados; relógio, lembretes, toast	`feat(US08): persiste agenda no localStorage` `feat(US09): adiciona lembretes com setInterval`
29/10	`feat/US11-node-modulos · fix/…`	Scripts npm, devDependency local, `fechamento-dia.js`, `validar-agenda.cjs`, `launch.json`; depurar e documentar o bug	`feat(US11): adiciona scripts de terminal ESM e CommonJS` `fix(US07): … · docs: registra bug-report`
30/10	`docs/readme-retrospectiva`	README completo (decisões, global vs local, CJS vs ESM, especificidade, retrospectiva), prints, Lighthouse, revisão do que veio de IA, ensaio da apresentação em live coding	`docs: conclui README e retrospectiva`

13. Plano de testes (cenários para você conferir)

#	Cenário	Esperado
T01	Abrir a página pelo `npm start`	7 serviços; nenhum erro vermelho no console
T02	Apontar o `fetch` do catálogo para um arquivo inexistente	Mensagem de erro + "Tentar novamente"
T03	Enviar o formulário vazio	Erros por campo na tela; foco no primeiro; nenhum `alert`
T04	Data de ontem · horário 07:00 · banho e tosa às 17:00 · horário 09:15	Cada um é rejeitado com mensagem clara (RN01)
T05	Salvar um agendamento válido	Aparece na agenda do dia; protocolo AP-0001; toast; formulário limpo; página não recarrega
T06	Domiciliar com CEP 01001-000	Rua "Praça da Sé", cidade São Paulo/SP; valor = preço + taxa
T07	CEP 99999-999 · CEP "123" · modo avião (DevTools → Network → Offline)	Três mensagens diferentes; preenchimento manual possível
T08	Serviço "Adestramento" + domiciliar	Caixa desabilitada com explicação
T09	Data 25/12 → depois 02/11	Aviso com o nome do feriado; salvar exige confirmação; **2ª consulta sem nova requisição** (aba Network)
T10	Escolher data de um ano ainda não consultado com a rede desligada	Aviso de indisponibilidade; consegue agendar
T11	Dois agendamentos no mesmo dia e hora · cancelar o primeiro · tentar de novo	1º conflito bloqueado; depois do cancelamento, aceito
T12	Buscar "sonia" (com "Sônia" salva) · filtrar por Cancelado · `Esc`	Busca ignora acento; filtro funciona; `Esc` limpa
T13	Teclas `/` e `N` fora e dentro de campos	Atalhos só funcionam fora dos campos
T14	Concluir um agendado	Status muda; botões desabilitados; fechamento atualiza; foco permanece no item
T15	F5 (recarregar)	Agenda, filtro e tema restaurados
T16	Console: `localStorage.setItem('agendapet:agenda', '{{ quebrado')` e F5	Página abre normalmente com agenda vazia
T17	Criar um agendamento para daqui a ~10 min · outro para 30 min atrás	Lembrete visível; o antigo marcado "Atrasado"

T18	Pausar e retomar lembretes várias vezes	Sem timers duplicados; botão e `aria-pressed` coerentes
T19	Trocar o tema e recarregar	Tema persiste; contraste ok nos dois
T20	"Limpar meus dados" (2 cliques)	1º clique pede confirmação; só chaves `agendapet:*` removidas
T21	Largura de 360, 768 e 1280 px	Sem rolagem horizontal; layout muda; botões fáceis de tocar
T22	Usar a página só com o teclado	Tudo alcançável; foco sempre visível
T23	Lighthouse (Acessibilidade e SEO)	≥ 90, ou pontos abaixo justificados no README
T24	`npm run fechamento` · `npm run validar` · `npm run validar:erro` · `echo $?`	Relatório correto; código 0; erros listados e código 1
T25	Clone limpo em outra pasta: `npm install` e `npm start`	Funciona sem nada além do README

14. Armadilhas frequentes (e dicas para revisar código de IA)

Armadilha	Como evitar
`fetch` não rejeita em 404/500	Sempre confira `resposta.ok` antes de `.json()`
ViaCEP responde 200 para CEP inexistente	Confira `dados.erro`
Esquecer `await` em `resposta.json()`	Ele também é Promise
Abrir o HTML com duplo clique	Módulos e `fetch` exigem servidor: `npm start`
Esquecer `preventDefault()` no submit	A página recarrega e tudo some
`novalidate` ausente com validação própria	O navegador mostra os balões dele antes das suas mensagens
`innerHTML` com texto do usuário	Use `textContent` (evita injeção de HTML/script)
Listener preso a elementos que são recriados	Delegação: um listener na lista + `closest()`
`setInterval` duplicado	Guarde o id; `clearInterval` antes de criar outro
Instância de classe no `localStorage`	Grave `paraJSON()`; restaure com a fábrica
`JSON.parse(null)` na 1ª visita / dado corrompido	Valor padrão + `try/catch`
`new Date("2026-10-20")` vira o dia anterior (UTC)	Use `"2026-10-20T09:00:00"` ou trabalhe com texto
Esquecer `super(...)` antes de usar `this`	Em subclasses, `super()` é a 1ª linha do construtor

`this` perdido em callback	Use arrow function ou métodos que não dependem de `this` externo
Limpar **todo** o `localStorage`	Apague só as chaves com o seu prefixo
Resposta atrasada sobrescrevendo dado novo	Compare o valor atual do campo com o da consulta antes de usar a resposta
`__dirname` em ESM / `import` estático em `.cjs`	`import.meta.url` no ESM; `await import()` no CJS
Estilo que "não pega"	Veja a especificidade no DevTools; troque seletor de tag por classe; nada de `!important`

Como revisar código de IA

(1) Rode e quebre de propósito (rede offline, campo vazio, storage corrompido). (2) Confira se usa só o que o curso ensinou ou se você sabe explicar cada linha. (3) Procure `var`, `innerHTML`, `!important`, bibliotecas e `alert`. (4) Confirme que a regra de negócio ficou em `js/dominio/`. (5) Anote para a apresentação o que a IA errou e como você corrigiu.

15. Checklist final de entrega

- ✓ Repositório **público**; `npm install` + `npm start` funcionam em um **clone limpo**
- ✓ Estrutura de pastas conforme a seção 2; `node_modules/` fora do repositório; `package-lock.json` versionado
- ✓ US01 a US12 atendem aos cenários de aceite do Documento Funcional
- ✓ Página semântica, um `h1`, foco visível, 360/768/1280 px sem rolagem horizontal, **sem** `!important` **e sem seletores de id**
- ✓ `fetch` com 3 estados e erros distintos; falha de API nunca bloqueia o trabalho
- ✓ Classes, herança (`extends` / `super`), `this`, closure, callback e ≥ 4 métodos de array; ao menos um laço
- ✓ `localStorage` com `try/catch`; limpar só o prefixo; timers com `clearInterval` / `clearTimeout`
- ✓ ESM (`export` / `import`) e um script CommonJS (`.cjs`); `npm run fechamento` e `npm run validar` funcionam
- ✓ `.vscode/launch.json` funcional; `docs/bug-report.md` completo + `docs/depuracao.png`
- ✓ Kanban com 4 colunas e 12 cartões estimados, com branch e PR em cada um; `planejamento/sprint-planning.md`
- ✓ ≥ 5 branches `tipo/US##`, ≥ 12 commits no padrão e ≥ 5 Pull Requests com o modelo preenchido
- ✓ README completo (decisões de negócio, global vs local, CJS vs ESM, especificidade, depuração, sprint e retrospectiva)
- ✓ Apresentação em live coding no Google Meet (sexta 30/10, 15 minutos por estudante, turma reunida), com IA declarada; links do repositório GitHub e do Kanban enviados no AVA até **02/11/2026, 22h**, com o código publicado no GitHub